



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:	JUEVES, 06/08/2026
Hora de inicio:	17:20

ANÁLISIS Y DISEÑO DE SISTEMAS 2 SECCIÓN B

Escuela de Ingeniería en Ciencias y Sistemas

ABDIEL FERNANDO JOSÉ OTZOY OTZÍN

/* UNIDAD 1 */

Patrones de diseño estructurales

01

Cómo ensamblar clases y objetos en estructuras flexibles y eficientes

Adapter

Proxy

Facade

Decorator

+ Bridge · Composite · Flyweight

¿Qué haremos hoy?

01

Hoja de calificación — Práctica 1

Lectura de los criterios y resolución de dudas antes de la entrega

02

Enunciado — Proyecto Fase 1

Lectura general del enunciado, alcance de la fase y preguntas

03

Horarios de calificación

Fechas y horarios para calificación de prácticas y proyecto

04

Patrones de diseño estructurales

Adapter · Proxy · Facade · Decorator — idea, UML, caso real y ejemplo en TypeScript

05

Cierre y despedida

Resumen comparativo, práctica en clase y referencias

// ANTES DE EMPEZAR

Objetivo de la clase

competencia

Identifica patrones de diseño analizando las necesidades de un problema específico, y selecciona el patrón que mejor se adapta a la solución propuesta.

al final de la clase podrás

- Reconocer cuándo aplicar **Adapter, Proxy, Facade y Decorator**
- Leer la estructura de cada patrón en un diagrama UML
- Implementar un ejemplo funcional en TypeScript
- Distinguir los cuatro patrones entre sí y no confundirlos

// valor del día

Disciplina

Capacidad de mantener hábitos de estudio, organización y esfuerzo constante, siguiendo normas y objetivos para alcanzar metas académicas y personales.

*/

Hoja de Calificación

idea y estructura → problema y solución → analogía y UML → caso real → ejemplo TypeScript

*/

Proyecto Fase 1

idea y estructura → problema y solución → analogía y UML → caso real → ejemplo TypeScript

*/

Horarios de calificación

idea y estructura → problema y solución → analogía y UML → caso real → ejemplo TypeScript

¿Qué es un patrón de diseño estructural?

definición

Explican **cómo ensamblar objetos y clases** en estructuras más grandes, manteniendo la flexibilidad y la eficiencia de esas estructuras.

analogía



Piensa en un juego de construcción tipo LEGO: cada patrón te da una forma específica de **conectar piezas** (clases y objetos) para construir algo complejo de manera organizada y reutilizable.

ruta de hoy

01 Adapter – traducir interfaces

02 Proxy – controlar acceso

03 Facade – simplificar subsistemas

04 Decorator – añadir funciones

mención breve:

Bridge • Composite • Flyweight

`/* PATRÓN 01 · ESTRUCTURAL */`

ADAPTER

Permite colaborar a objetos con interfaces incompatibles.

idea y estructura → problema y solución → analogía y UML → caso real → ejemplo TypeScript

01

La idea: un traductor entre interfaces

Adapter permite la colaboración entre objetos con interfaces incompatibles: envuelve uno de los objetos y traduce sus llamadas, sin modificar su código.

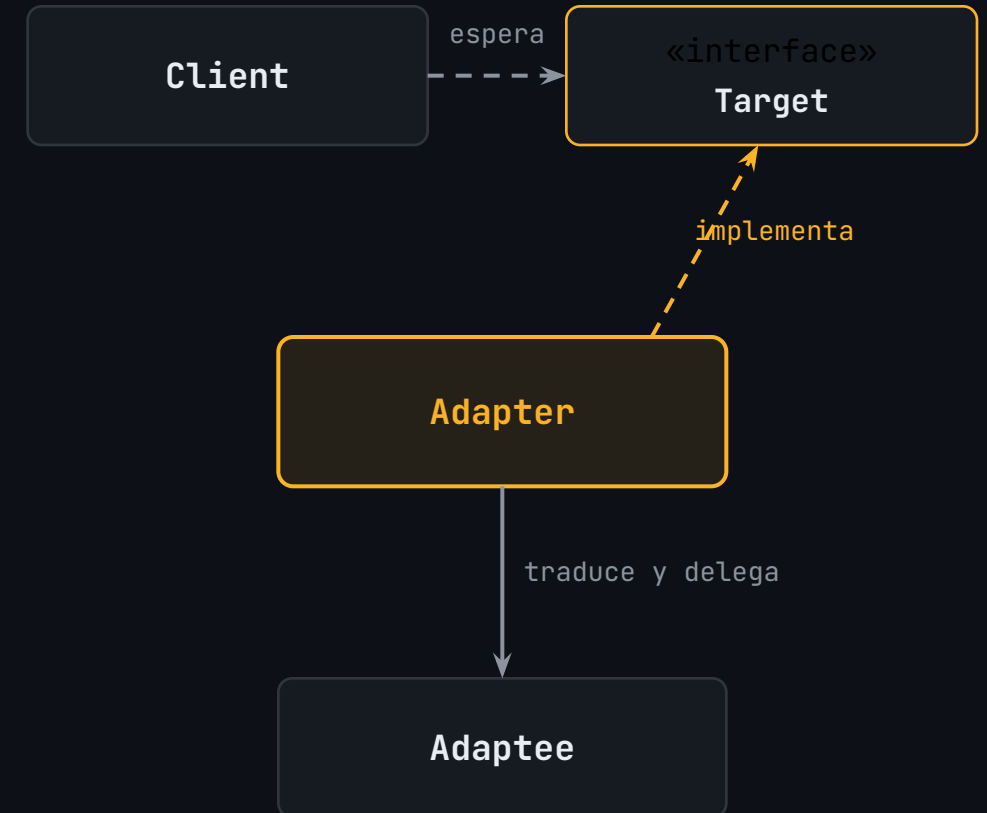
los 4 roles

Client — tu código; solo sabe trabajar con la interfaz Target.

Target — la interfaz que el cliente espera (el contrato).

Adaptee — la clase existente con la interfaz incompatible; normalmente de un tercero y no la puedes modificar.

Adapter — implementa Target, contiene al Adaptee y traduce cada llamada de uno al otro.



// el Adaptee no sabe que el Adapter existe

Del problema a la solución

PROBLEMA

Tu app de monitoreo bursátil descarga datos en formato **XML** y los muestra con gráficos.

Quieres integrar una biblioteca de análisis de terceros... pero solo entiende **JSON**.

- ✗ Modificar la biblioteca puede romper el código que ya depende de ella.
- ✗ Y puede que ni siquiera tengas acceso a su código fuente.

```
tu_app      → XML
biblioteca  → JSON  # incompatible
```

SOLUCIÓN

Crea un **adaptador XML → JSON** por cada clase de la biblioteca que uses. Tu código habla solo con los adaptadores; cada uno traduce los datos y delega la llamada.

1. El adaptador expone la interfaz que tu app espera.
2. Tu app invoca al adaptador con datos XML.
3. El adaptador convierte a JSON y llama a la biblioteca.

```
app -XML→ [Adapter] -JSON→ biblioteca ✓
```

Analogía y estructura UML



Viajas de Europa a EE. UU. y tu cargador **no entra en el enchufe** : las clavijas son distintas. No cambias el cargador ni la instalación eléctrica: usas un adaptador de viaje.

cada pieza de la analogía es un rol

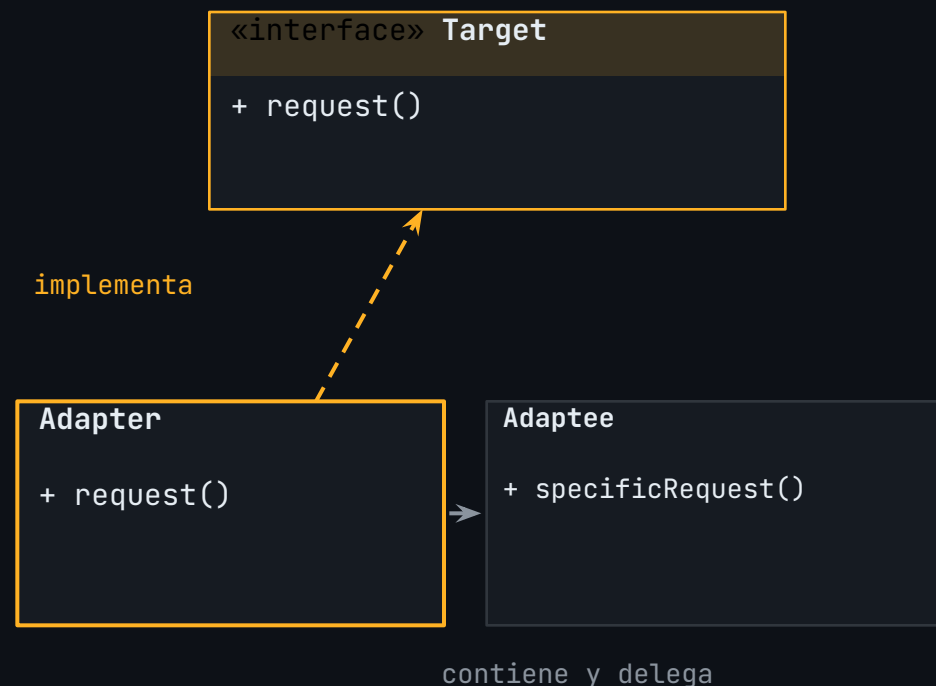
Tu laptop → **Client** : necesita energía y solo entiende su clavija.

Clavija europea → **Target** : la interfaz que espera.

Tomacorriente americano → **Adaptee** : existe, funciona, pero es incompatible.

Adaptador de viaje → **Adapter** : traduce una forma a la otra.

diagrama de clases



El Adapter recibe `request()` y la convierte en `specificRequest()` sobre el Adaptee.

Caso real: dos APIs bancarias, una sola interfaz

contexto

Dos bancos exponen APIs para **aprobar créditos personales**. Hacen prácticamente lo mismo, pero con interfaces distintas: sin patrón, necesitarías dos implementaciones separadas.

Con un adaptador por banco, tu sistema expone una única interfaz de crédito y oculta las diferencias de cada API.

- ✓ Menos código duplicado: un solo flujo de préstamos.
- ✓ Si mañana llega un tercer banco, solo escribes otro adaptador.
- ✓ Tu lógica de negocio no conoce los detalles de cada API.

flujo de una llamada

- 1 El cliente llama a `request()` en el Adapter
↓
- 2 El Adapter traduce formato y orden de los datos
↓
- 3 Delega en `specificRequest()` del Adaptee
↓
- 4 El Adaptee ejecuta el trabajo real
↓
- 5 El resultado regresa por el Adapter...
↓
- 6 ...y el cliente lo recibe en el formato que espera

Pasarela de pago vieja, interfaz nueva

● ● ● pago.ts

```
interface IPago {          // lo que tu app espera
  cobrar(monto: number): boolean;
}

class OldGateway {         // pasarela vieja: intocable
  processPayment(value: string): "OK" | "FAIL" { /* ... */ }
}

class OldGatewayAdapter implements IPago {
  private gateway = new OldGateway();

  cobrar(monto: number): boolean {
    if (monto ≤ 0) return false; // regla extra
    const value = monto.toFixed(2); // number → string
    return this.gateway.processPayment(value) === "OK";
  }
}

const pago: IPago = new OldGatewayAdapter();
pago.cobrar(199.99); // → processPayment("199.99") → true
```

La tarea

Crea OldGatewayAdapter que implemente IPago. Dentro de cobrar(monto):

1. Convierte el monto a string con 2 decimales.
2. Llama a processPayment.
3. Devuelve true si responde "OK".

Regla extra: si el monto es ≤ 0 , devuelve false sin llamar a la pasarela.

criterio de aceptación

El resto del sistema solo usa IPago. Al inyectar el adaptador, cobrar(199.99) termina llamando processPayment("199.99").

`/* PATRÓN 02 · ESTRUCTURAL */`

PROXY

Un sustituto que controla el acceso a otro objeto.

idea y estructura → problema y solución → analogía y UML → caso real → ejemplo TypeScript

02

La idea: un intermediario con la misma interfaz

Proxy actúa como sustituto de otro objeto para **controlar el acceso** a él. El cliente habla con el proxy; el proxy delega en el objeto real en el momento y la forma correctos.

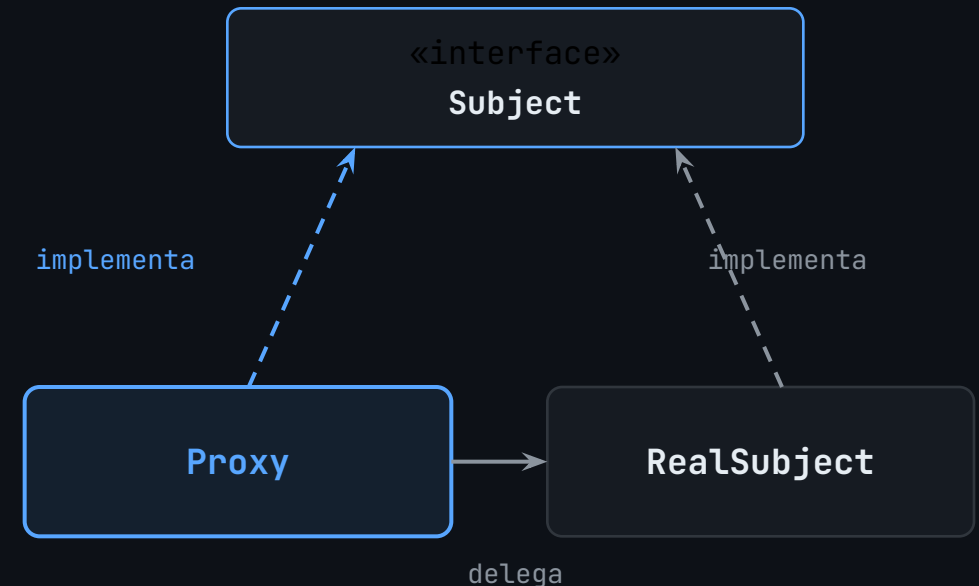
los 4 roles

Subject — interfaz común que implementan el objeto real y el proxy.

RealSubject — el objeto pesado o sensible que hace el trabajo real.

Proxy — mantiene una referencia al RealSubject y puede añadir lógica antes o después de delegar: caché, seguridad, carga diferida...

Client — usa el Subject sin saber (ni importarle) que habla con un proxy.



Misma interfaz en ambos: el cliente puede intercambiar Proxy y RealSubject **sin cambiar una línea** de su código.

Del problema a la solución

PROBLEMA

Tienes un **objeto enorme** que consume muchos recursos... pero solo lo necesitas de vez en cuando.

Podrías crearlo solo cuando haga falta (*inicialización diferida*), pero entonces **todos los clientes** tendrían que repetir ese código de inicialización.

- ✗ Código duplicado por toda la aplicación.
- ✗ Y si la clase es de una biblioteca cerrada, no puedes meter esa lógica dentro.

SOLUCIÓN

Crea una clase **proxy** con la misma interfaz que el objeto original y entrégala a todos los clientes. Al recibir una petición, el proxy crea el objeto real **solo cuando hace falta** y le delega el trabajo.

la ventaja

¿Necesitas ejecutar algo **antes o después** de la lógica principal — validar permisos, cachear resultados, registrar auditoría?

El proxy lo hace sin tocar la clase original, y puede pasarse a cualquier cliente que espere el servicio real.

Analogía y estructura UML



Una **tarjeta de crédito** es un proxy de tu cuenta bancaria — y la cuenta, un proxy del efectivo. Todas sirven para *pagar* (misma interfaz), pero pagar con tarjeta añade control: no cargas billetes, hay límites y registro de cada transacción.

cada pieza de la analogía es un rol

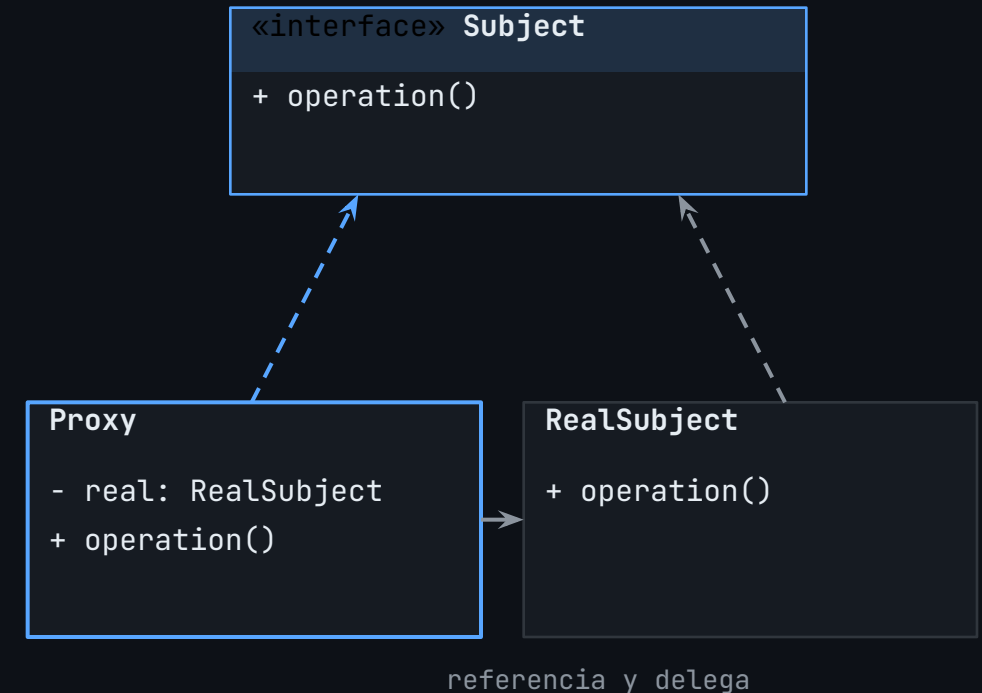
"Pagar" → **Subject**: la operación que todos entienden.

Cuenta bancaria → **RealSubject**: donde está el dinero de verdad.

Tarjeta de crédito → **Proxy**: controla y registra el acceso a la cuenta.

El comercio → **Client**: cobra sin saber cómo está guardado el dinero.

diagrama de clases



Ambos implementan Subject: el proxy es **intercambiable** por el objeto real ante el cliente.

Caso real: seguridad y auditoría transparentes

contexto

Un sistema ejecuta **procesos sensibles** (pagos, cierres de caja, reportes). No todos los usuarios pueden ejecutarlos.

Un proxy de protección intercepta cada ejecución: valida los privilegios del usuario **antes** de delegar y registra la ejecución **después** (auditoría).

- ✓ Los procesos reales no cambian: la seguridad vive en el proxy.
- ✓ El usuario no nota la capa extra: misma interfaz.
- ✓ Queda un registro de quién ejecutó qué y cuándo.

flujo de una ejecución

- 1 El usuario llama a un método en el Proxy
- 2 El Proxy valida privilegios (seguridad, caché, lazy)
- 3 ¿Control exitoso? Sí → delega · No → acceso denegado
- 4 El objeto real ejecuta el trabajo
- 5 El Proxy registra la ejecución (auditoría)
- 6 El cliente recibe el resultado

Todo el control ocurre **antes y después** de delegar: el objeto real solo se entera de hacer su trabajo.

Galería en línea con carga diferida

```
● ● ● galeria.ts

interface Imagen { mostrar(): void; }
class RealImage implements Imagen {
  constructor(private archivo: string) {
    this.cargarDelServidor(); // lento y pesado
  }
  mostrar() { /* render en alta resolución */ }
}
class ImageProxy implements Imagen {
  private real?: RealImage; // caché
  constructor(private archivo: string) {}
  mostrar() {
    this.real ??= new RealImage(this.archivo);
    this.real.mostrar(); // 2ª vez: usa la caché
  }
}
const foto: Imagen = new ImageProxy("foto-4k.png");
foto.mostrar(); // aquí recién se carga del servidor
```

el encargo

Las imágenes de la galería son de **muy alta resolución** y se cargan lento. El cliente interactúa con un ImageProxy:

1. Muestra un placeholder de inmediato.
2. Carga la imagen real solo cuando el usuario la pide (*lazy loading*).
3. Guarda en caché la imagen cargada.

qué observar

RealImage se crea **una sola vez** : en el primer `mostrar()`. Las llamadas siguientes usan la instancia cacheada y responden al instante.

`/* PATRÓN 03 • ESTRUCTURAL */`

FACADE

03

Una interfaz simple y unificada para un subsistema complejo.

idea y estructura → problema y solución → analogía y UML → caso real → ejemplo TypeScript

La idea: una sola puerta de entrada

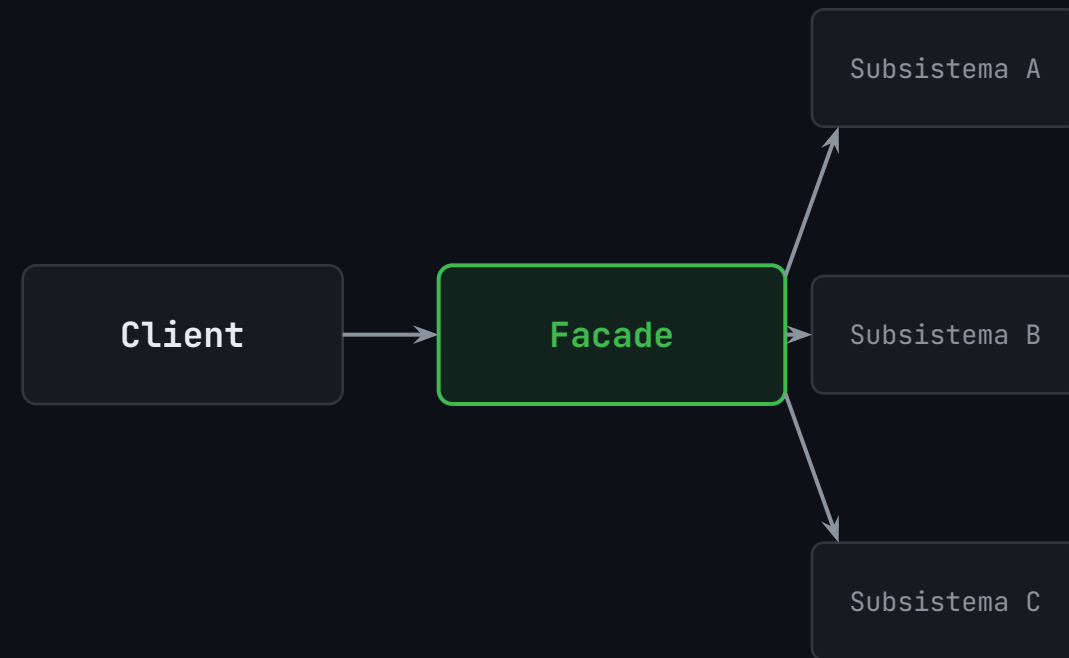
Facade ofrece una interfaz simple y unificada a un subsistema complejo: oculta sus clases internas y expone **un único punto de entrada** con las operaciones que el cliente realmente necesita.

los 3 roles

Facade — la clase con la interfaz simplificada; internamente delega en los objetos correctos del subsistema.

Subsystem classes — las clases de bajo nivel que hacen el trabajo real; el cliente nunca habla directo con ellas.

Client — solo conoce la fachada; ignora la complejidad interna.



Una llamada simple del cliente se reparte entre varios subsistemas **sin que él lo sepa**.

Del problema a la solución

PROBLEMA

Tu código debe trabajar con un **grupo amplio de objetos** de una biblioteca o framework sofisticado.

Eso implica inicializar objetos, rastrear dependencias y ejecutar métodos **en el orden correcto** .

- × Tu lógica de negocio queda acoplada a detalles de terceros.
- × El código se vuelve difícil de entender y mantener.

SOLUCIÓN

Una **fachada** es una clase con una interfaz simple hacia el subsistema. Ofrece menos funciones que el subsistema completo, pero **justo las que los clientes necesitan** .

Ideal cuando integras una biblioteca con decenas de funciones de las que solo usas una pequeña parte.

ejemplo

Tu app sube videos de gatos a redes sociales y usa una biblioteca profesional de conversión de video...

Pero solo necesitas `codificar(archivo, formato)`. Esa clase conectada a la biblioteca **ya es tu primera fachada** .

Analogía y estructura UML



Llamas por teléfono a una tienda para hacer un pedido. El **operador** es tu fachada: con una simple conversación accedes al sistema de pedidos, la pasarela de pago y el servicio de entrega — sin hablar con cada departamento.

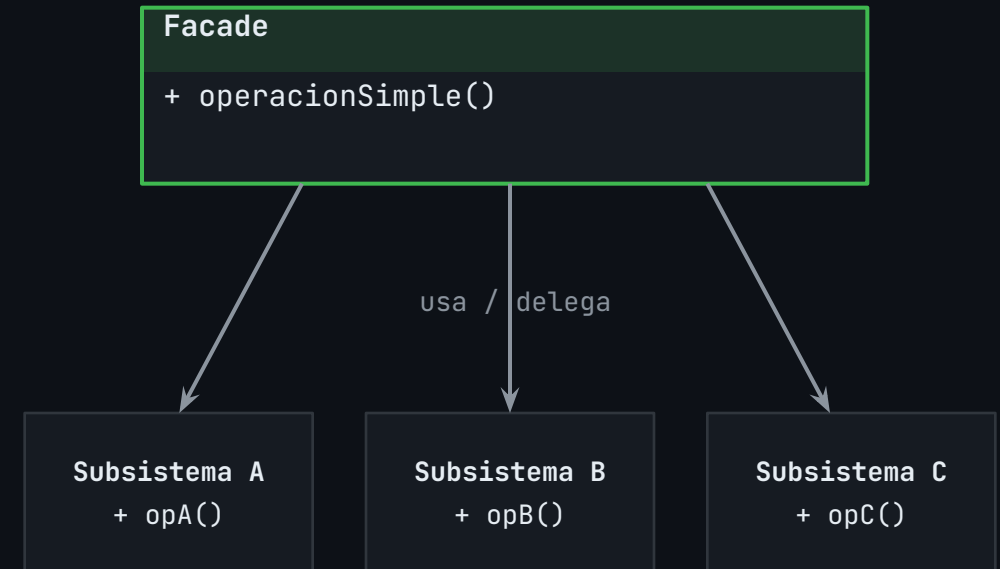
cada pieza de la analogía es un rol

El operador → **Facade**: interfaz de voz simple.

Pedidos, pagos, envíos → **Subsystem classes**: departamentos que hacen el trabajo.

Tú, el cliente → **Client**: dices qué quieres; ignoras cómo se coordina todo por dentro.

diagrama de clases



La fachada **no reemplaza** al subsistema: si hace falta, aún puedes usar las clases internas directamente.

Caso real: pagos en línea sin dolor

contexto

Cobrar en línea implica coordinar **varios sistemas** : validar la tarjeta, verificar fondos, aplicar el cargo, generar el recibo y notificar al usuario.

Pedirle a cada programador que entienda todos esos subsistemas es inviable. Una fachada de pagos expone operaciones de alto nivel y se encarga de la coordinación interna.

- ✓ Los programadores usan operaciones simples, sin conocer el subsistema.
- ✓ Si un subsistema cambia, solo se ajusta la fachada.
- ✓ El orden correcto de los pasos vive en un solo lugar.

flujo de un pago

- 1 El cliente llama a `procesarPago()` en la fachada
↓
- 2 La fachada valida la tarjeta (Subsistema A)
↓
- 3 Verifica fondos y aplica el cargo (Subsistema B)
↓
- 4 Genera el recibo y notifica (Subsistema C)
↓
- 5 Consolida los resultados de cada paso
↓
- 6 El cliente recibe una respuesta simple: aprobado o rechazado

AuthFacade: registro y login en dos llamadas

● ● ● auth.ts

```
class AuthFacade {
  private validador = new Validador();
  private hasher    = new Hasher();
  private repo      = new Repositorio();
  private tokens    = new TokenService();
  async register(email: string, pass: string) {
    this.validador.emailValido(email);
    const hash = this.hasher.hash(pass);
    return this.repo.guardar({ email, hash });
  }
  async login(email: string, pass: string) {
    const user = this.repo.buscar(email);
    this.hasher.verificar(pass, user.hash);
    return { token: this.tokens.emitir(user.id) };
  }
}

// el cliente solo ve la fachada:
await auth.register("ana@mail.com", "secreto");
await auth.login("ana@mail.com", "secreto");
```

el encargo

Registrar y loguear usuarios implica validar email, "hashear" la contraseña, guardar el usuario y emitir un token.

La fachada expone solo:

register() · login()
getProfile() · logout()

qué observar

El cliente llama **dos métodos** ; la fachada coordina **cuatro subsistemas** en el orden correcto. Los subsistemas quedan ocultos.

`/* PATRÓN 04 · ESTRUCTURAL */`

DECORATOR

04

Añade funcionalidades a un objeto envolviéndolo, sin tocar su código.

idea y estructura → problema y solución → analogía y UML → caso real → ejemplo TypeScript

La idea: capas que envuelven al objeto

Decorator permite añadir funcionalidades a un objeto de forma dinámica, envolviéndolo en objetos "decorador" que implementan **la misma interfaz** . El cliente no nota la diferencia.

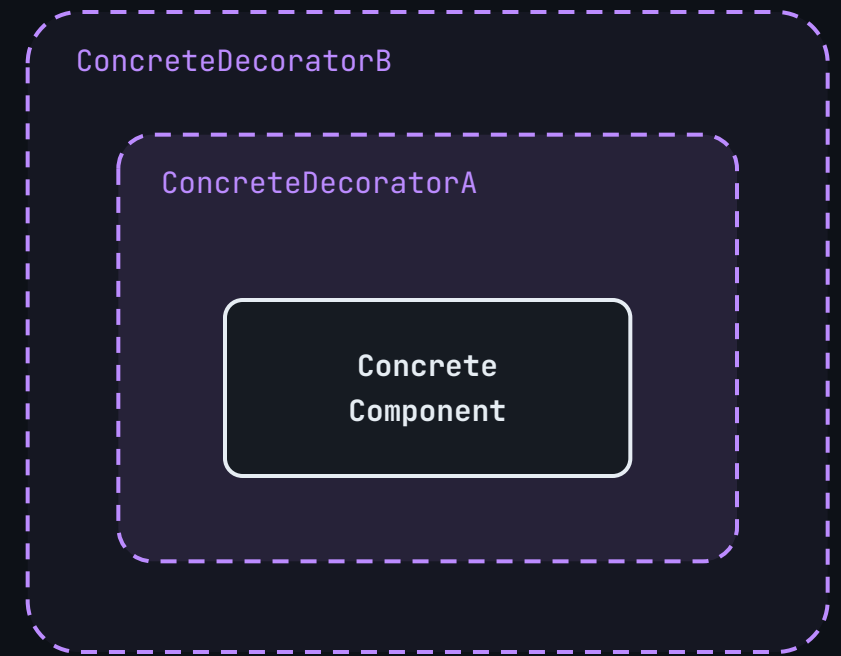
los 4 roles

Component — la interfaz común del objeto base y de los decoradores.

ConcreteComponent — el objeto original que queremos "decorar".

Decorator — clase abstracta que implementa Component y guarda una referencia al objeto envuelto.

ConcreteDecorator — añade su funcionalidad antes o después de delegar en el objeto envuelto.



Cada capa añade algo y delega hacia adentro. Las capas se **combinan y reordenan** en tiempo de ejecución.

Del problema a la solución

PROBLEMA

Tu biblioteca de notificaciones enviaba solo **correos** . Ahora los usuarios piden SMS, Facebook y Slack.

Creas subclases... y alguien pregunta: *"¿por qué no varios canales a la vez?"*

- ✗ Combinar canales con herencia genera una subclase por combinación: una explosión de clases.
- ✗ La herencia es estática: no puedes cambiar el comportamiento en tiempo de ejecución.

Email+SMS • Email+Slack • SMS+Slack • Email+SMS+Slack ...

SOLUCIÓN

Sustituye herencia por **composición** : un objeto guarda una referencia a otro y le delega parte del trabajo.

Cada canal se vuelve un **decorador** que envuelve al notificador base. Apílalos en cualquier combinación, en tiempo de ejecución.

así se apila

```
SlackDecorator(  
    SMSDecorator(  
        EmailNotifier() ))
```

// 3 canales, 0 subclases nuevas

Analogía y estructura UML



Vestirte por capas es decorar: si tienes frío, te pones un **suéter** ; si sigue el frío, una **chaqueta** encima; si llueve, un **impermeable** . Cada prenda "extiende" tu comportamiento básico... y puedes quitarla cuando quieras.

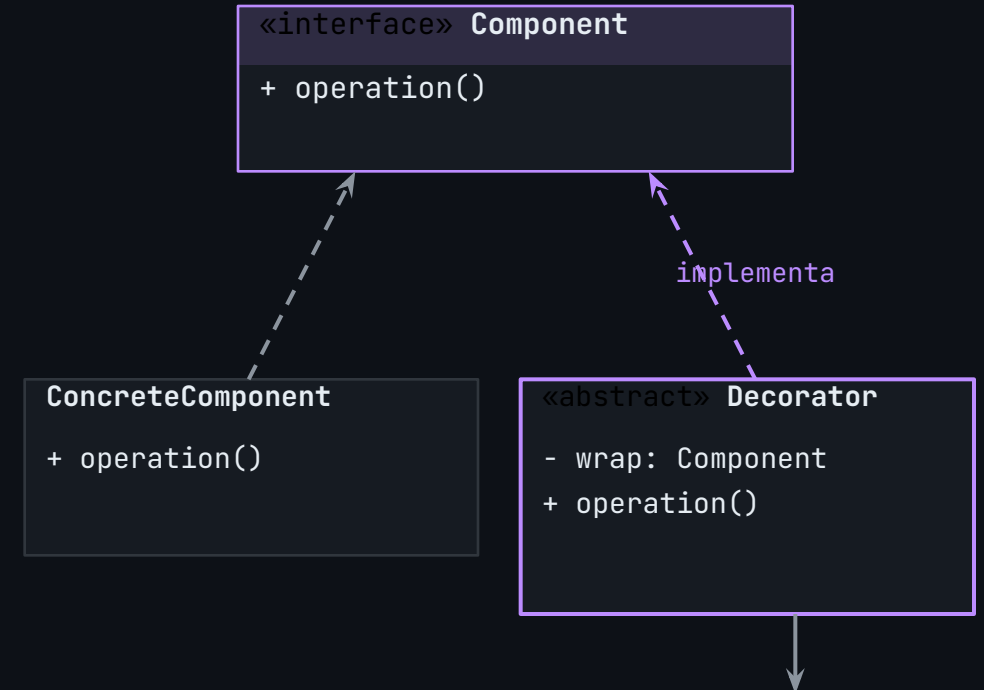
cada pieza de la analogía es un rol

Tú → **ConcreteComponent** : el objeto base.

"Abrigarse" → **Component** : la interfaz que todo cumple.

Suéter, chaqueta, impermeable → **ConcreteDecorators** : capas que se añaden y quitan en cualquier orden.

diagrama de clases



El Decorator **guarda un Component** dentro: por eso los decoradores pueden envolver otros decoradores y apilarse.

Caso real: un mensaje procesado por capas

contexto

Un mensaje debe viajar seguro: primero se convierte el objeto a **XML**, luego se envuelve en un sobre **SOAP** y al final se **encripta**.

Cada paso es un **decorador** que recibe el resultado del anterior y le añade su transformación.

- ✓ Cambiar el orden de las capas produce un resultado distinto.
- ✓ Se pueden insertar nuevas capas (compresión, firma digital) en medio del proceso.
- ✓ Cada capa se prueba y entiende por separado.

flujo de transformación

1 Objeto de negocio — el mensaje original



2 **XMLDecorator** — convierte el objeto a XML



3 **SoapDecorator** — envuelve el XML en un sobre SOAP



4 **EncryptDecorator** — mensaje SOAP encriptado, listo para enviar

Cafetería: precios que se componen solos

● ● ● cafe.ts

```
interface Beverage { costo(): number; }

class Coffee implements Beverage {
  costo() { return 10; }
}

abstract class BeverageDecorator implements Beverage {
  constructor(protected wrap: Beverage) {}
  abstract costo(): number;
}

class Milk extends BeverageDecorator {
  costo() { return this.wrap.costo() + 2; }
}

class Sugar extends BeverageDecorator {
  costo() { return this.wrap.costo() + 1; }
}

const pedido = new Sugar(new Milk(new Coffee()));
pedido.costo(); // 10 + 2 + 1 = 13
```

el **encargo**

Una tienda de café en línea tiene la clase base Beverage y opciones como Milk, Sugar o Soy.

Combínalas **sin modificar la clase base** : café simple, café con leche, o café con leche y azúcar — y calcula el costo total.

qué **observar**

Cada costo() delega hacia adentro y **suma su extra**. Añadir Soy es crear otra clase decorador: la base nunca cambia.

Otros patrones estructurales que existen



Bridge

Divide una clase grande en dos jerarquías separadas — abstracción e implementación — que evolucionan de forma independiente.

// úsalo cuando una jerarquía crece en dos dimensiones a la vez



Composite

Compone objetos en estructuras de árbol y trabaja con ellas como si fueran objetos individuales (una carpeta y un archivo se tratan igual).

// úsalo cuando tu modelo es un árbol: menús, archivos, organigramas



Flyweight

Caben más objetos en la RAM compartiendo el estado común entre ellos, en lugar de repetir la misma información en cada uno.

// úsalo cuando creas miles de objetos parecidos: partículas, caracteres, fichas

Hoy los reconoces por nombre; si tu problema encaja con uno, ya sabes **dónde buscar** .

¿Cuándo uso cada patrón?

PATRÓN	QUÉ HACE	ÚSALO CUANDO...	EJEMPLO DE HOY
Adapter	Traduce una interfaz a otra para que dos objetos incompatibles colaboren	...integras código de terceros que no puedes modificar	Pasarela de pago vieja → IPago
Proxy	Controla el acceso a un objeto: caché, seguridad, carga diferida	...necesitas añadir control sin tocar el objeto real	Galería con lazy loading
Facade	Simplifica un subsistema complejo con una interfaz unificada	...tu código depende de muchas clases de una biblioteca	AuthFacade: register / login
Decorator	Añade funcionalidades envolviendo el objeto, en tiempo de ejecución	...las combinaciones por herencia explotarían en subclases	Café + leche + azúcar

// regla rápida: traduce • controla • simplifica • extiende

Practiquemos lo aprendido



Genera una **analogía del patrón Adapter** con un objeto de la vida real (distinto al enchufe) e identifica en ella las partes que conforman la solución del patrón.

cómo trabajarlo

1. Elige un objeto cotidiano que "conecte" dos cosas incompatibles.
2. Nombra quién es el **Client**, cuál es la interfaz **Target**, qué hace de **Adaptee** y qué hace de **Adapter**.
3. Comparte tu analogía con la clase.

criterio de logro

- ✓ Los 4 roles están identificados y son coherentes.
- ✓ El Adapter "traduce" algo real: formato, idioma, medida, voltaje...
- ✓ Ni el Client ni el Adaptee cambian: solo se interpone el adaptador.

// pista: traductor de idiomas, cargador USB-C → Lightning, adaptador de llaves de tuercas...

 tutor@ads2:~/unidad-1\$ exit --clase-completada

¡Gracias por su atención!

Nos vemos en la próxima clase — repasa los ejemplos y prepara tus dudas del proyecto.

// [referencias](#)

→ Refactoring.Guru — Patrones estructurales (guía con ejemplos): refactoring.guru/es/design-patterns/structural-patterns

→ Los ejemplos de clase están disponibles en el material del curso.